Gabriel Martinez
WES 237C
Project 3 - DFT

Project 3 – DFT Report

Questions.

Question 1: DFT32 Baseline Implement a baseline HLS implementation for dft_32. You may use the cos() and sin() functions from the math library.

(a) Now think about if you were to use a custom CORDIC algorithm to calculate cos() and sin() (you don't have to implement this). Would changing the accuracy of your CORDIC core make the DFT hardware resource usage change? How would it affect the performance?

Answer. Yes. Increasing CORDIC accuracy by performing more rotations increases latency and reduces throughput, while the hardware resource usage remains nearly constant. In my Project 2 results, increasing rotations from 10 to 20 increased latency from 144 to 274 cycles, while LUT and DSP usage remained unchanged. Thus, changing the accuracy of the CORDIC core would make the DFT faster or slower without significantly increasing hardware resource usage.

Question 2: DFT32 Table Lookup Rewrite the code to eliminate these math function calls (i.e. *cos()* and *sin()*) by utilizing a table lookup. Use the provided **_2D.h* file.

(a) Make a table that shows the change in resource utilization and performance with between Question 1 and 2.

Answer.

Resource Utilization	Q1: cos()/sin()	Q2: Table lookup
LUT	18,024	91,730
FF	14,756	59,208
DSP	220	640
BRAM	18	66
Latency (cycles)	96,964 - 103,108	257
Initiation interval (cycles)	96,965 - 103,109	258

Target Clock (ns)	10	10
Estimated Clock period (ns)	7.297	7.256
Throughput (MHz)	.001329 - .001413	.5362

Question 3: DFT32 Interface Change Modify the DFT function interface so that the input and outputs are stored in separate arrays. Modify the testbench to accommodate this change to DFT interface.

(a) Why did we do this? Does it affect what optimizations you can perform?

Answer. We separated the input and output arrays to eliminate the need to overwrite the original input data removing the last copy loop. This makes optimizations such as pipelining, loop unrolling, and array partitioning easier by reducing memory dependencies.

(b) Make a table that shows the resource utilization and performance from before and after this change.

Answer.

Resource	Before (Q2)	After (Q3)
LUT	91,730	91,521
FF	59,208	59,189
DSP	640	640
BRAM18K	66	64
Latency (cycles)	257	221
Interval (cycles)	258	222
Target Clock (ns)	10	10
Estimated clock (ns)	7.256	7.256
Throughput (MHz)	.5362	.6234

(c) Describe the results you see.

Answer. Separating the input and output arrays reduced latency from 257 to 221 cycles and increased throughput by ~16.2%. Resource usage decreased slightly, and DSP usage remained unchanged.

Question 4: DFT32 Array Partitioning Experiment with array partitioning. Partition all arrays in your implementation. For now, make sure that your loops are being pipelined with a target II of 1.

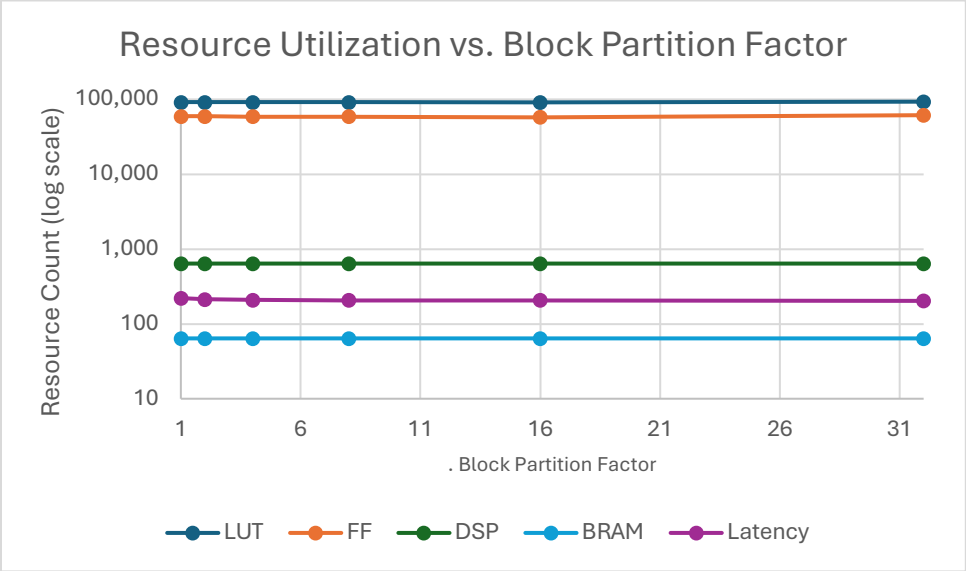
(a) Use block partitioning. Try factors of 1 (i.e. without partitioning), 2, 4, 8, 16, and 32. Make a table showing the achieved II, resource utilization, and performance of each of these implementations.

Answer.

Block factor	#pragma HLS PIPELINE II = 1	LUT	FF	DSP	BRAM	Latency (cycles)	Throughput (MHz)	Estimated Clock (ns)
1	1	91,521	59,189	640	64	221	0.623	7.256
2	1	91,675	59,314	640	64	213	0.647	7.256
4	1	91,669	59,054	640	64	209	0.659	7.256
8	1	91,705	58,540	640	64	207	0.666	7.256
16	1	91,252	57,515	640	64	206	0.669	7.256
32	1	93,250	61,509	640	64	203	0.679	7.256

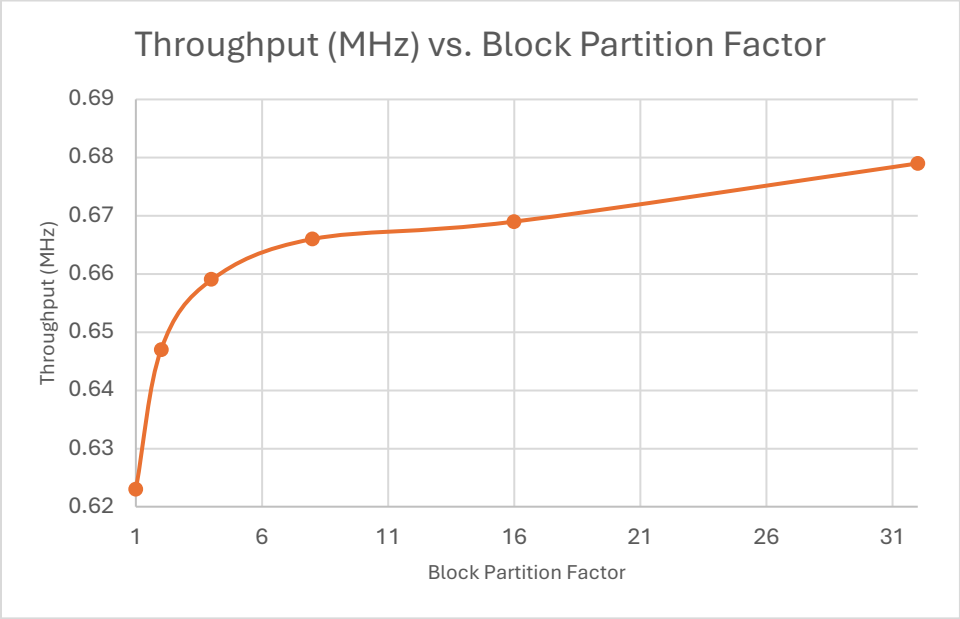
(b) Plot resource utilization vs the partition factor on one plot.

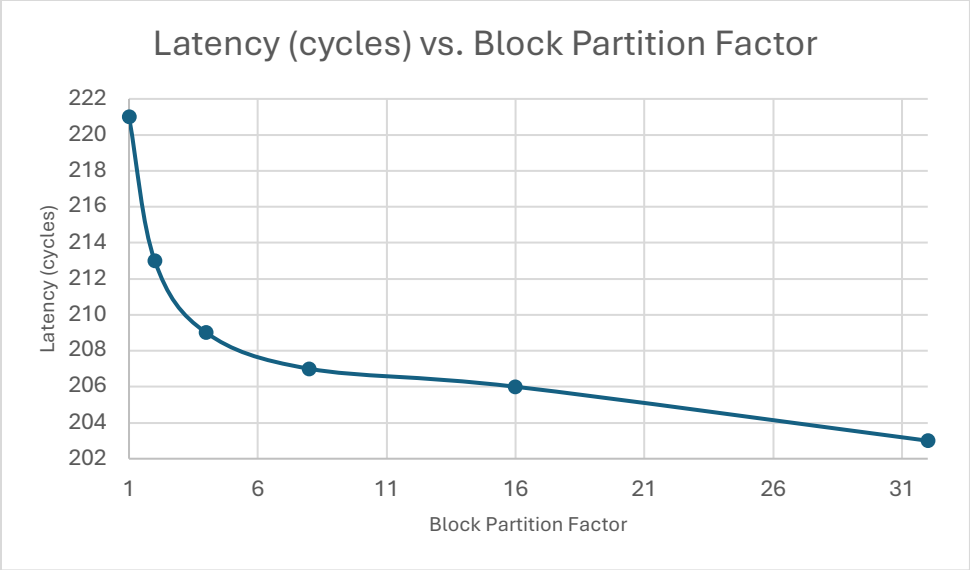
Answer.



(c) Plot throughput & latency vs the partition factor on separate plots.

Answer.





Question 5: DFT32 Loop Unrolling Experiment with loop unrolling. Unroll the inner loop only. Use the best array partitioning from Question 4. In order to prevent automatic unrolling by Vitis HLS, you will need to remove the #pragma HLS PIPELINE pragmas from both loops. Double check that your explicit unrolling is being applied by looking at the synthesis report.

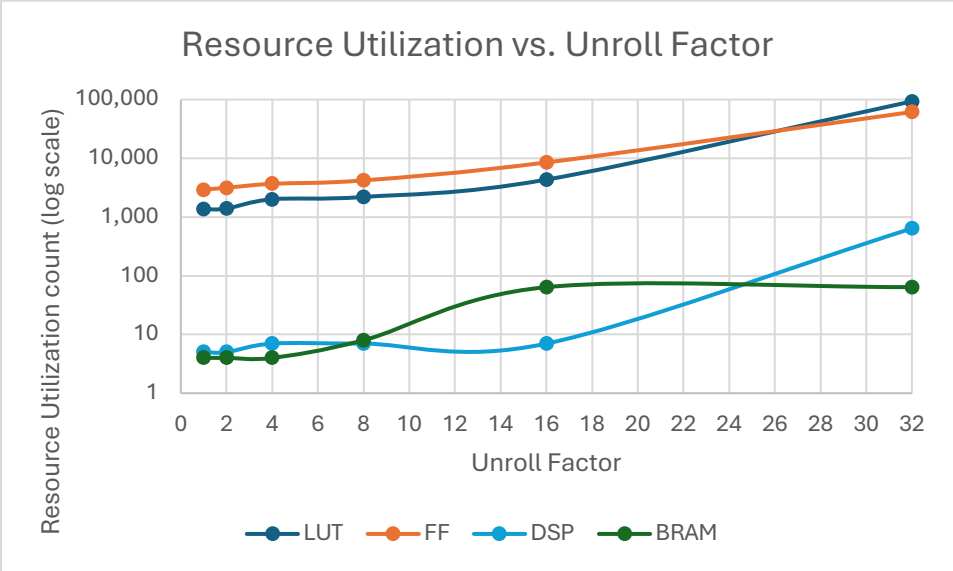
(a) Try factors of 1 (i.e. without unrolling), 2, 4, 8, 16, and 32. Make a table showing the achieved II, resource utilization, and performance of each of these implementations.

Answer.

Unroll factor	Initiation Interval	LUT	FF	DSP	BRAM18K	Latency (cycles)	Throughput (MHz)	Estimated Clock (ns)
1	6	1,351	2,923	5	4	6,721	0.0205	7.256
2	11	1,396	3,157	5	4	6,209	0.0222	7.256
4	21	2,009	3,681	7	4	5,953	0.0198	8.495
8	41	2,199	4,212	7	8	5,825	0.0185	9.305
16	81	4,339	8,507	7	64	5,761	0.0181	9.591
32	1	93,250	61,509	640	64	203	0.6789	7.256

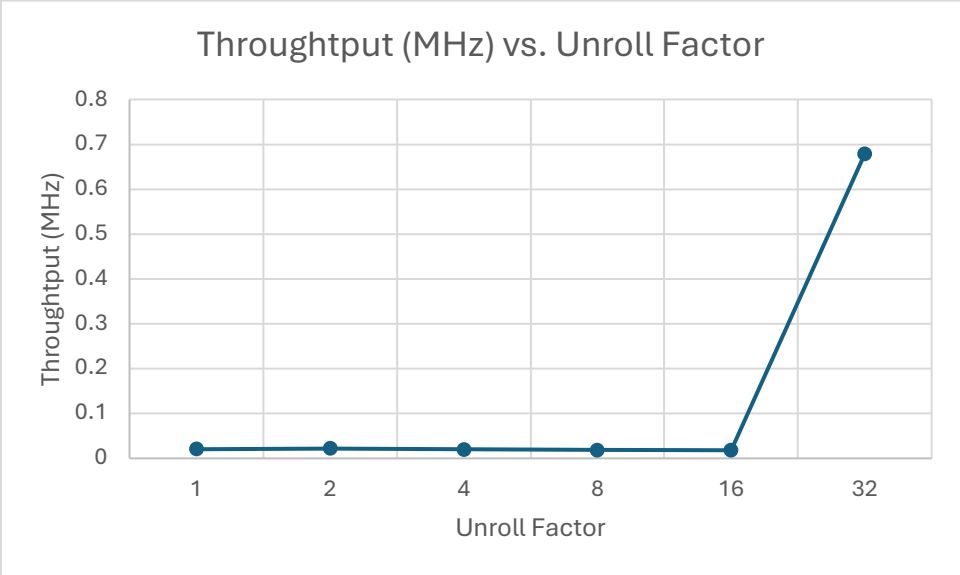
(b) Plot resource utilization vs the partition factor on one plot.

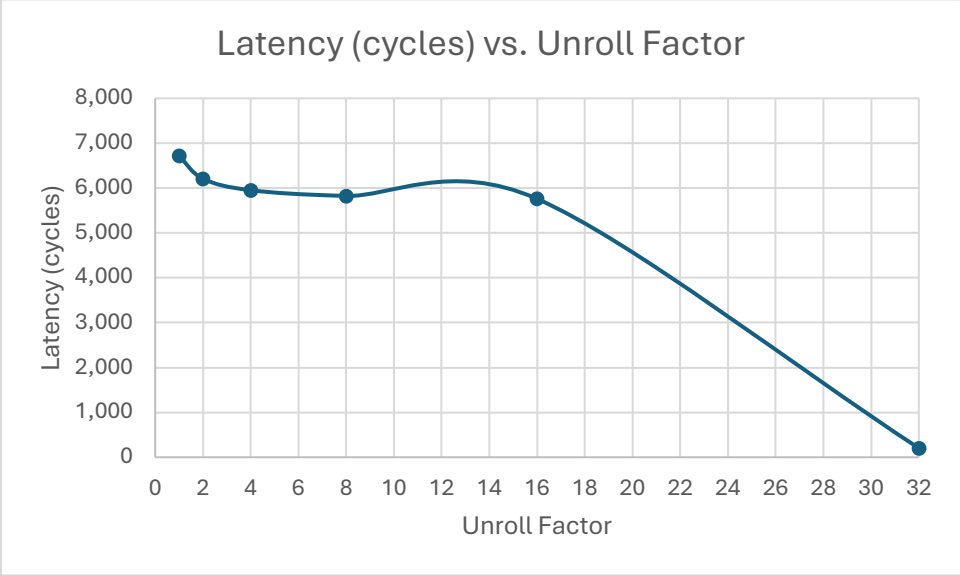
Answer.



(c) Plot throughput & latency vs the partition factor on separate plots.

Answer.





Question 6: DFT1024 Baseline You should refer to the baseline DFT code at Figure 4.15 of the textbook.

(a) Write a basline DFT1024 using the sin() and cos() math functions. Do not apply any HLS pragmas. Report latency, throughput, and resource utilization.

Answer.

Resource	Baseline
Latency (cycles)	98,588,676 - 104,880,132
Estimated Clock (ns)	7.297
Throughput (Hz)	1.3067 - 1.39
LUT	18,012
FF	14,791
DSP	220
BRAM18K	16

(b) A full 2D lookup table no longer fits on a PYNQ-Z2 board. We can only store a 1D array of precomputed sin() and cos() values. Re-write the baseline DFT1024 to use the pre-computed values. Report the latency, throughput and resource utilization.

Answer.

Resource	Baseline	Table Lookup
Latency (cycles)	98,588,676-104,880,132	6,292,502
Estimated Clock (ns)	7.297	7.256
Throughput (Hz)	1.3067 - 1.39	21.8975
LUT	18,012	1,605
FF	14,791	1,247
DSP	220	5
BRAM18K	16	4

Question 7: DFT1024 Loop optimization The baseline DFT1024 from Figure 4.15 of the textbook has data dependencies in the inside loop, which could limit parallelism. One way to tackle this issue is to interchange the two loops. Implement this change and report the latency, throughput, and resource.

Resource	Q6(a) Baseline	Q7 Loop Interchange
Latency (cycles)	98,588,676 - 104,880,132	114,304,007 - 120,595,463
Estimated Clock (ns)	7.297	7.297
Throughput (Hz)	1.3067 - 1.39	1.1364 - 1.1990
LUT	18,012	18,146
FF	14,791	15,008
DSP	220	220
BRAM18K	16	16

Question 8: DFT1024 Best Design: Now that you have explored different optimizations for DFT32, we can go ahead and try their ideas on DFT1024.

(a) Try any optimization techniques and describe your methodology.

Answer. I optimized the DFT1024 using precomputed sine and cosine lookup tables instead of calculating trigonometric functions during execution, and the inner loop was partially unrolled using #pragma HLS UNROLL factor=2 to increase parallelism.

(b) Report the latency, throughput, and resource utilization of your best design. Your design must fit on the PYNQ-Z2 board, which mean all resource utilizations must be less than 100%.

Answer.

Resource	Q6(a) Baseline	Q6(b) Table Lookup	Q7 Loop Interchange	Q8(b) Optimized (Unroll 2)
Latency (cycles)	98,588,676 - 104,880,132	6,292,502	114,304,007 - 120,595,463	5,768,214
Estimated Clock (ns)	7.297	7.297	7.297	7.256
Throughput (Hz)	1.3067 - 1.39	21.8975	1.1364 - 1.1990	23.891
LUT	18,012	1,605	18,146	1,640
FF	14,791	1,247	15,008	1,432
DSP	220	5	220	5
BRAM18K	16	4	16	4

Question 9: Streaming Interface Synthesis: Modify your design to allow for streaming inputs and outputs using hls::stream. You must write your own testbench to account for the function interface change from DTYPE to proper hls::stream. You can learn about hls::stream from the HLS Stream Library. You should also follow the Lab: Axistream Multiple DMAs example. Report the latency, throughput, and resource utilization of your design. Resource utilization must be under 100%. Using your optmized DFT1024 is optional, you can also just convert the baseline DFT1024 to streaming interface.

Answer.

Resource	Q9
Latency (cycles)	5,768,216
Estimated Clock (ns)	7.256
Throughput (Hz)	23.8909
LUT	1,597
FF	1,288
DSP	5
BRAM18K	8

GitHub Link: https://github.com/Math140Instructor/wes237c/tree/main/project3

Demo video:
https://drive.google.com/file/d/1MlvPI_DmGHYazCS1C0QcQY5QIktBzFGa/view?usp=sharing